Incorporating X-ray Microscopy into Self-Driving Laboratories

Gordon Research Conference: AI For Materials, Energy, and Chemical Sciences 2026

Nathan S. Johnson
Carl Zeiss Research Microscopy Solutions
5300 Central Parkway, Dublin, CA 94568
nathan.johnson@zeiss.com

1 Introduction

Self-driving laboratories have emerged as a powerful paradigm for accelerating scientific discovery by tightly coupling experiment execution, data analysis, and decision-making in automated closed-loop workflows. While substantial progress has been made in automating synthesis, simulation, and data-driven optimization, advanced microscopy remains a persistent bottleneck. In particular, X-ray microscopy requires expert knowledge to safely configure instrument geometry, select acquisition parameters, center samples, and interpret imaging results. These tasks are traditionally performed manually and are difficult to encode using fixed-rule automation alone.

This document presents a technical description of a multi-agent large-language-model (LLM) control system developed to autonomously operate a ZEISS Versa 630 X-ray microscope. The system is designed as a modular microscopy control layer that can be integrated into broader self-driving laboratory frameworks. Unlike monolithic LLM control approaches, the system emphasizes explicit orchestration, deterministic hardware interfaces, structured state, and layered memory, enabling safe, repeatable, and extensible autonomous microscopy.

2 Software Architecture Overview

The system is implemented in Python and organized around four tightly integrated subsystems: (1) a LangGraph-based multi-agent controller, (2) MCP-exposed deterministic hardware and geometry services, (3) a layered retrieval-augmented generation (RAG) memory system, and (4) a metrics, logging, and profiling framework. The architecture separates reasoning, execution, and state management, which is essential for safe autonomy on physical laboratory equipment.

At runtime, an interactive driver initializes an isolated workspace, constructs an initial `AgentState`, and executes a compiled LangGraph application until the workflow terminates. All intermediate artifacts, metadata sidecars, and logs are written to a structured directory hierarchy, enabling reproducibility and post hoc inspection.

Major Python packages include `langgraph` (state-machine orchestration), `langchain` (model abstraction and message types), `fastmcp` (MCP servers and typed tool schemas), `chromadb`/`langchain_chroma` (vector stores), and scientific Python libraries (NumPy, OpenCV/scikit-image where applicable) for image processing. Instrument control relies on the ZEISS XradiaPy API, wrapped behind MCP tools to ensure deterministic, auditable behavior.

3 Control-Flow Diagram

Figure 1 summarizes the primary LangGraph control flow. The key design pattern is that execution proceeds *agent* → *world update* → *verifier*, with the verifier routing back to the same agent for batch continuation, retry, or escalation to the supervisor for replanning.

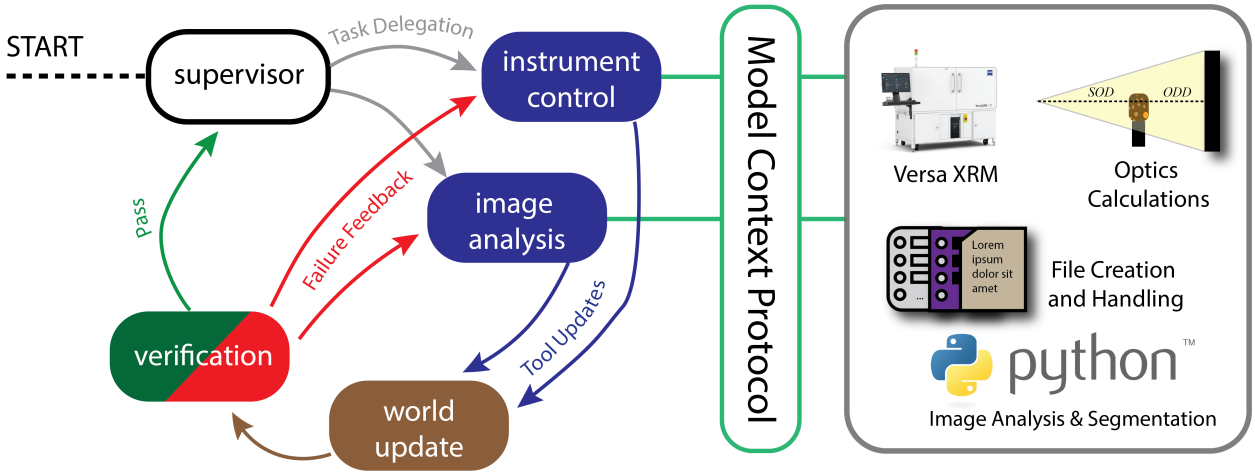

Figure 1: High-level control flow of the LangGraph application. The supervisor plans and assigns task batches. Instrument and image agents execute MCP tool calls, producing a tool span. A world-update node reduces tool outputs into a structured patch (world model, artifacts, trace, lessons). A verifier node evaluates completion, handles retries, and routes control to the next node via `state['next']`.

4 Agent State Model

All nodes in the graph operate over a shared mutable `AgentState` dictionary. This state is the authoritative working memory of the system and is designed to minimize dependence on long conversational history. Table 1 summarizes the major fields.

5 LangGraph Topology and Agent Roles

The LangGraph application consists of five nodes: **Supervisor**, **Instrument Agent**, **Image Agent**, **World Update**, and **Verifier**. The supervisor performs task planning and batch assignment. The instrument agent executes microscope control actions via MCP (motion, objective/filter selection, acquisition). The image agent executes analysis tasks (segmentation, centroid extraction, transmission computation) and geometry reasoning via deterministic geometry tools.

After agent tool execution, the world-update node reduces the raw tool span into a structured patch, merging updates into `world_model`, indexing any produced artifacts, and appending task trace entries. The verifier then evaluates whether the current task is complete, failed, or requires retry, and sets `state['next']` accordingly.

A critical performance optimization is **batch execution**: the supervisor assigns a list of task IDs to an agent, and the agent is instructed to execute all tasks in sequence. Batch continuation

is implemented by verifier routing back to the same agent (agent → world update → verifier → agent) until the batch is exhausted, rather than forcing a supervisor intervention after each tool call.

6 Formal Pseudocode for the Agent Loop

Algorithm 1 formalizes the control loop implemented by the LangGraph state machine. The pseudocode focuses on the design pattern in which (i) the supervisor emits tasks and assignments, (ii) agents execute tools via MCP, (iii) tool outputs are reduced into structured state patches, and (iv) a verifier gate controls retries and routing.

7 Deterministic Tool Execution via MCP

A central safety and robustness feature is that LLM agents do not directly manipulate the microscope. Instead, agents can only invoke a curated tool surface exposed via MCP. Tools are capability-scoped (e.g., instrument motion, acquisition, geometry, image processing, file handling). Tool calls are executed by a tool runner (e.g., a `ToolNode`) which (i) injects the working directory and run context, (ii) executes the tool with typed arguments, and (iii) returns structured tool outputs.

The system includes explicit error handling for two classes of failures: tool-reported failures (detected by scanning tool outputs for error keywords) and tool-execution exceptions. For exceptions, the system synthesizes placeholder tool messages to preserve the strict one-to-one pairing between tool calls and tool responses required by tool-calling protocols, ensuring the downstream reducer and verifier can always operate on a valid tool span.

8 Layered Retrieval-Augmented Memory System

Reliable long-horizon autonomy in microscopy requires that the system retain, retrieve, and apply knowledge at multiple timescales and levels of abstraction. To this end, the controller implements a layered memory architecture that combines explicit working memory, agent-specific episodic memory, and shared contextual memory. This architecture is implemented in `rag_manager.py` and is tightly integrated with the LangGraph controller, world-update logic, and verifier gate.

At the lowest level, the system maintains an explicit *working memory* encoded directly in the shared `AgentState`. This includes the authoritative world model (instrument configuration, sample state, safety constraints), the task list and task trace, recent tool spans, retry counters, and short-term *lessons* distilled from recent successes and failures. Working memory is deterministic, fully inspectable, and injected verbatim into every agent prompt. Unlike conversational history, it is not subject to stochastic summarization and therefore serves as the primary grounding mechanism for agent reasoning across long workflows.

Above working memory, the system implements *episodic memory*, which captures procedural knowledge accumulated from prior workflows. Episodic memory is agent-specific: separate persistent ChromaDB vector stores are maintained for the supervisor, instrument agent, and image agent. Each episodic memory entry is encoded as a structured trigger–procedure pair, where the trigger describes a task context (e.g., "center sample in the field of view") and the procedure encodes a detailed, step-by-step operational recipe.

Episodic memory entries are stored as structured documents with explicit metadata (agent, trigger, type) and embedded using OpenAI or Azure OpenAI embedding models. At runtime, agents

query their respective episodic stores using the current task description, retrieving the top-k most relevant procedures. Retrieved procedures are injected into the agent prompt as explicit system instructions, biasing the LLM toward previously validated workflows while preserving flexibility.

Importantly, episodic memory functions as a *soft policy layer*. Many entries encode operational constraints and safety rules, such as retry limits for X-ray source activation, mandatory ordering of stage movements, prohibitions against reimplementing geometry calculations in free-form Python, and strict routing rules between agents. This allows the system to enforce best practices and safety constraints without hard-coding brittle logic into the controller.

The third layer is *contextual memory*, which captures shared scientific and technical knowledge that is not specific to a single workflow instance. Contextual memory is implemented as a single shared ChromaDB vector store and is intended to hold domain knowledge such as imaging physics, segmentation strategies, or empirically validated acquisition parameters for particular material classes. Long documents are automatically split into overlapping chunks using a recursive text splitter before embedding, enabling fine-grained semantic retrieval.

Contextual memory queries are explicitly gated and typically invoked only for analysis-heavy reasoning, such as selecting segmentation strategies, interpreting transmission measurements, or reasoning about sample-class-specific imaging trade-offs. Retrieval is governed by explicit hyperparameters, including top-k limits and keyword gating, to prevent uncontrolled prompt growth. Retrieved content is formatted and grouped by source before injection into agent prompts.

Taken together, this layered memory architecture enables the system to combine deterministic state tracking, experiential procedural knowledge, and domain context within a single coherent reasoning framework. By clearly separating working memory from long-term vectorized memory, the controller avoids the brittleness of purely conversational context while retaining the adaptability required for autonomous X-ray microscopy.

9 Hyperparameters and Operational Knobs

Key operational hyperparameters include the model/provider choice, LangGraph recursion limits, batching policy, maximum retries, transcript retention limits, pruning thresholds (message count and token budget), post-pruning target token limits, and RAG parameters (episodic and contextual top-k, contextual keyword gates). Capability gating flags control which tool domains are exposed to each agent, providing an additional safety boundary.

10 Demonstration Workflows

Demonstrated workflows include instrument setup, single- and two-view centering, transmission correction, and tomography preparation. In centering workflows, images are acquired, segmented, and converted into stage-space corrections using geometry services based on acquisition sidecar metadata. Two-view centering uses orthogonal projections to ensure alignment in three-dimensional stage coordinates. Tomography workflows extend this loop to magnification selection and projection acquisition, producing datasets suitable for reconstruction and downstream analysis.

11 Performance and Benchmarking

Benchmark studies across multiple model configurations isolate the effect of planning quality on cost, reliability, runtime, and token consumption while holding architecture constant. Results indicate that planning fidelity dominates system-level efficiency; weaker models inflate tokens and

runtime due to retries and replanning, while stronger models converge more quickly despite higher per-call costs. MCP-constrained execution ensures that failures remain safe and recoverable.

12 Threats to Validity

Benchmarks were conducted on a single representative workflow and may not capture behavior under substantially longer horizons or adversarial inputs. Performance depends on orchestration and prompt design choices, and cost and latency are subject to external API variability. Nevertheless, the qualitative trends observed—particularly the dominance of planning quality over actuation fidelity—are expected to generalize across similar autonomous microscopy deployments.

State key	Type	Description / semantics
`messages`	list[Message]	Bounded transcript (Human/AI/Tool messages). Older messages may be pruned/summarized to enforce token and count budgets.
`world_model`	dict	Authoritative structured working memory (e.g., `sample`, `instrument`, `safety`, `session`). Injected into every agent prompt.
`tasks`	list[dict]	Task list. Each task typically includes `task_id`, `description`, `assigned_to`, `status`, and optional `result`.
`assigned_tasks_for_agent`	list[str]	Current batch of task IDs assigned to an agent. Used to reduce LLM overhead via batched execution.
`last_tool_span`	dict	Captured tool span (AI tool-call message + ToolMessages) for post-processing by world update.
`task_trace`	dict	Per-task structured trace (tool calls, results, timestamps, verifier decisions). Enables auditability and debugging.
`artifacts_index`	dict	Registry of generated files (images, sidecars, recon outputs) with metadata linking artifacts to tasks and sample/session context.
`lessons`	list[dict]	Short-term experiential memory distilled by world-update (e.g., "what worked" / "what failed"). Capped and injected into prompts.
`memory_context`	str	Retry hints / additional context injected on failure (e.g., last error summary, suggested correction).
`last_error`	str	Most recent detected error string (tool failure or exception). Used by verifier and supervisor.
`retry_count`	int	Global or per-workflow retry counter; incremented by verifier on failure.
`max_retries`	int	Maximum retries before escalation/abort (also configurable globally).
`next`	str	Routing directive used by LangGraph conditional edges (e.g., `instrument`, `image`, `world_update`, `supervisor`, `FINISH`).
`decision_log`	list[dict]	Timestamped log of supervisor routing and rationale for traceability.

Table 1: Representative `AgentState` schema used by the LangGraph controller. Exact key set may include additional workflow-specific fields (e.g., workspace paths, metrics handles).

Algorithm 1 LangGraph-based multi-agent control loop (simplified)

Input: user request u, configuration $\mathcal{C}$
Initialize workspace $\mathcal{W}$; initialize state $S \leftarrow \textsc{CreateInitialState}(\mathcal{W}, u, \mathcal{C})$
$S[\texttt{next}] \leftarrow \texttt{supervisor}$
while $S[\texttt{next}] \neq \texttt{FINISH}$ **do**
  if $S[\texttt{next}] = \texttt{supervisor}$ **then**
    $S \leftarrow \textsc{PruneIfNeeded}(S, \mathcal{C})$
    $ctx \leftarrow \textsc{InjectMemory}(S, \texttt{supervisor}, \mathcal{C})$ ▷ RAG + lessons + world model
    $plan \leftarrow \text{LLM\_Plan}(u, S, ctx)$ ▷ JSON: tasks + assignments + routing
    $S \leftarrow \textsc{ApplyPlan}(S, plan)$
    $S[\texttt{next}] \leftarrow plan.\texttt{next}$
  else if $S[\texttt{next}] \in \{\texttt{instrument}, \texttt{image}\}$ **then**
    $agent \leftarrow S[\texttt{next}]$
    $S \leftarrow \textsc{PruneIfNeeded}(S, \mathcal{C})$
    $ctx \leftarrow \textsc{InjectMemory}(S, agent, \mathcal{C})$
    $taskBatch \leftarrow S[\texttt{assigned\_tasks\_for\_agent}]$
    $toolCalls \leftarrow \text{LLM\_Act}(agent, taskBatch, S, ctx)$
    $toolMsgs \leftarrow \textsc{ExecuteToolsViaMCP}(toolCalls, \mathcal{W})$
    $S[\texttt{last\_tool\_span}] \leftarrow \textsc{CaptureSpan}(toolCalls, toolMsgs)$
    $S[\texttt{next}] \leftarrow \texttt{world\_update}$
  else if $S[\texttt{next}] = \texttt{world\_update}$ **then**
    $patch \leftarrow \text{LLM\_}\textsc{ReduceToolSpan}(S[\texttt{last\_tool\_span}], S)$
    $S \leftarrow \textsc{DeepMerge}(S,\ patch.\texttt{world\_model\_patch})$
    $S \leftarrow \textsc{IndexArtifacts}(S,\ patch.\texttt{artifacts})$
    $S \leftarrow \textsc{UpdateTaskTrace}(S,\ patch.\texttt{task\_trace\_patch})$
    $S \leftarrow \textsc{AppendLesson}(S,\ patch.\texttt{lesson})$
    $S[\texttt{next}] \leftarrow \texttt{verifier}$
  else if $S[\texttt{next}] = \texttt{verifier}$ **then**
    $decision \leftarrow \text{LLM\_}\textsc{Verify}(S)$ ▷ JSON: done/fail/retry + routing
    $S \leftarrow \textsc{ApplyVerifierDecision}(S, decision)$
    if $decision.\texttt{status} = \texttt{failed}$ **then**
      $S[\texttt{retry\_count}] \leftarrow S[\texttt{retry\_count}] + 1$
      if $S[\texttt{retry\_count}] > S[\texttt{max\_retries}]$ **then**
        $S[\texttt{next}] \leftarrow \texttt{supervisor}$ ▷ escalate / replan / abort
      else
        $S[\texttt{memory\_context}] \leftarrow decision.\texttt{retry\_hint}$
        $S[\texttt{next}] \leftarrow decision.\texttt{retry\_route}$ ▷ instrument or image
      end if
    else
      $S[\texttt{next}] \leftarrow decision.\texttt{next}$ ▷ batch continue or supervisor
    end if
  end if
end while
Finalize workspace $\mathcal{W}$; export metrics, artifacts, and logs
Return: final state S